Multilayer Perceptrons, Part 2

Advanced Machine Learning

Sam Ogden

DRAFT

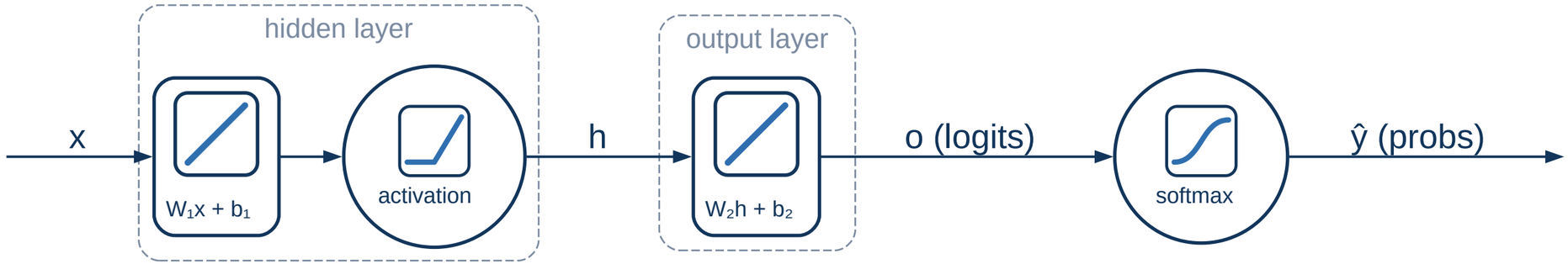
This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- See where each local derivative lives along the model, then chain them
- Follow the error backward one neuron at a time, into a hidden layer
- Walk the full backward pass and one SGD update, on the example from last time

Recap: the forward pass

Where we left off



Same fruit classifier, same example: $x = [1.2,\ 0.2]$, true class banana.

stage	value
$h_{\text{pre}} = W_1 x + b_1$	$[1.4,\ -0.8]$
$h = \text{ReLU}(h_{\text{pre}})$	$[1.4,\ 0]$
$o = W_2 h + b_2$	$[0.7,\ -0.42,\ 0.28]$
$\hat{y} = \text{softmax}(o)$	$[0.5042,\ 0.1645,\ 0.3313]$
$L = -\log \hat{y}_{\text{banana}}$	1.8048

The Big Question

The loss only touches W_1 and b_1 *indirectly* — how do we figure out how wrong they were?

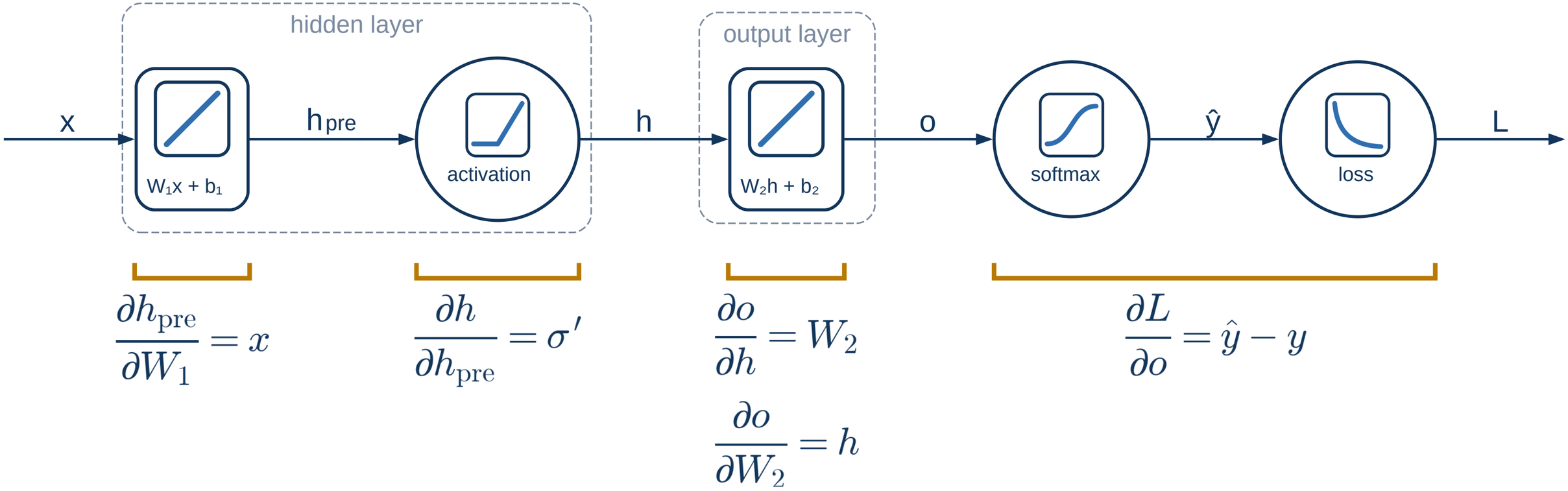
Gradients: the rough picture

Which derivative lives where

The Big Idea

The chain rule is the key to this – we just follow each step backwards through the network

Local derivatives along the model



- Each bracket marks one stretch of the model and the one small derivative for it — computed from only what's inside that stretch

Chain them together

$$\frac{\partial L}{\partial W_2} = \underbrace{\frac{\partial L}{\partial o}}_{\hat{y}-y} \underbrace{\frac{\partial o}{\partial W_2}}_{h}$$

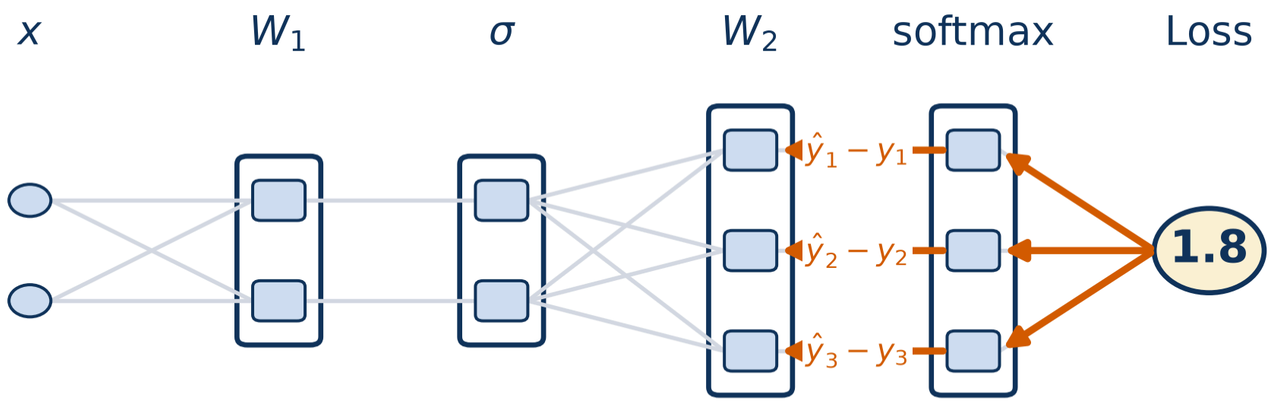
$$\frac{\partial L}{\partial W_1} = \underbrace{\frac{\partial L}{\partial o}}_{\hat{y}-y} \underbrace{\frac{\partial o}{\partial h}}_{W_2} \underbrace{\frac{\partial h}{\partial h_{\text{pre}}}}_{\sigma'} \underbrace{\frac{\partial h_{\text{pre}}}{\partial W_1}}_{x}$$

- Multiply the local derivatives along the path from the loss back to the weight
- W_2 is one link from the loss; W_1 is three links away — same rule, more pieces
- Read these as a map: the next slides work out exactly how the pieces multiply, one neuron at a time

Gradients: the in-depth picture

One neuron, one weight at a time

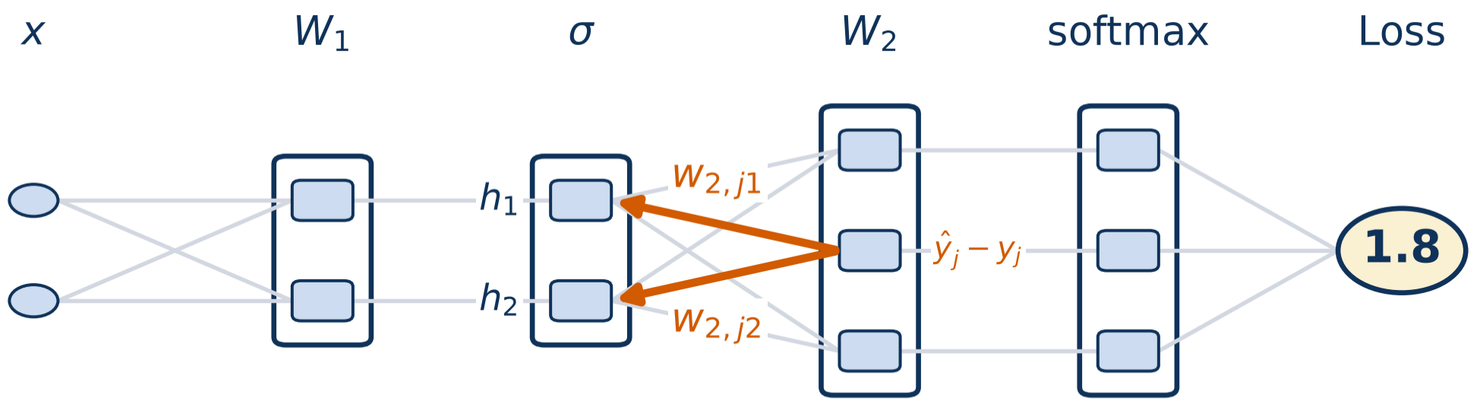
Where the error starts



$\frac{\partial L}{\partial o_j} = \hat{y}_j - y_j$ for each output neuron j (softmax and cross-entropy together)

- Backpropagation starts at the loss and moves backward, one stretch at a time
- Softmax and cross-entropy together hand each output neuron its error, $\hat{y}_j - y_j$ (derived last lecture). Nothing between here and the loss has weights to train, so the learning happens further back

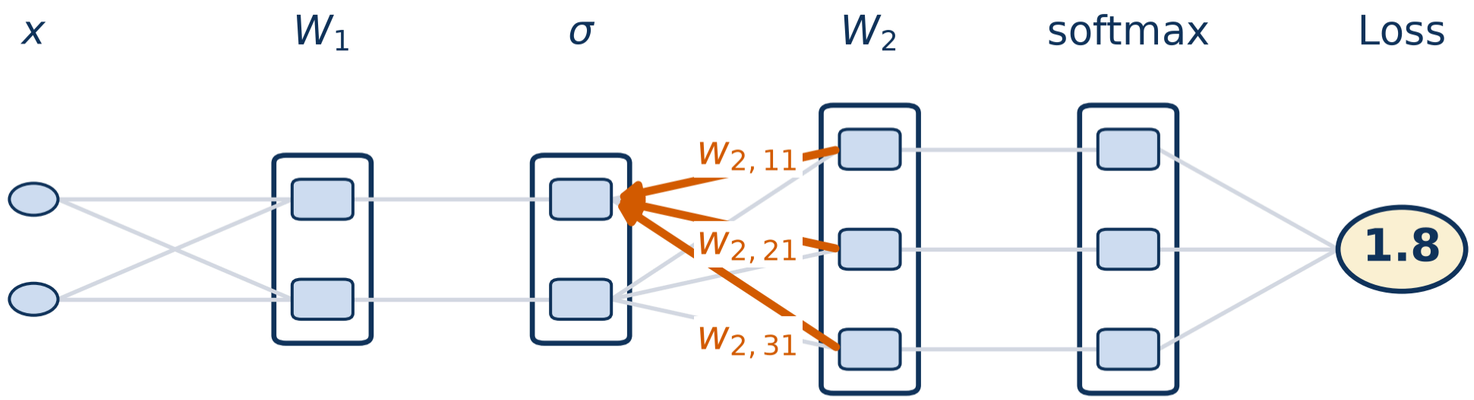
The output layer's weights



$$\frac{\partial L}{\partial w_{2,jk}} = \frac{\partial L}{\partial o_j} \cdot \frac{\partial o_j}{\partial w_{2,jk}} = (\hat{y}_j - y_j)\, h_k$$

- The error at output neuron j has already arrived: $\dfrac{\partial L}{\partial o_j} = \hat{y}_j - y_j$
- $w_{2,jk}$ (row j, column k of W_2) connects h_k to output neuron j, and multiplies it: $\dfrac{\partial o_j}{\partial w_{2,jk}} = h_k$. That gives last lecture's formula, with h for x

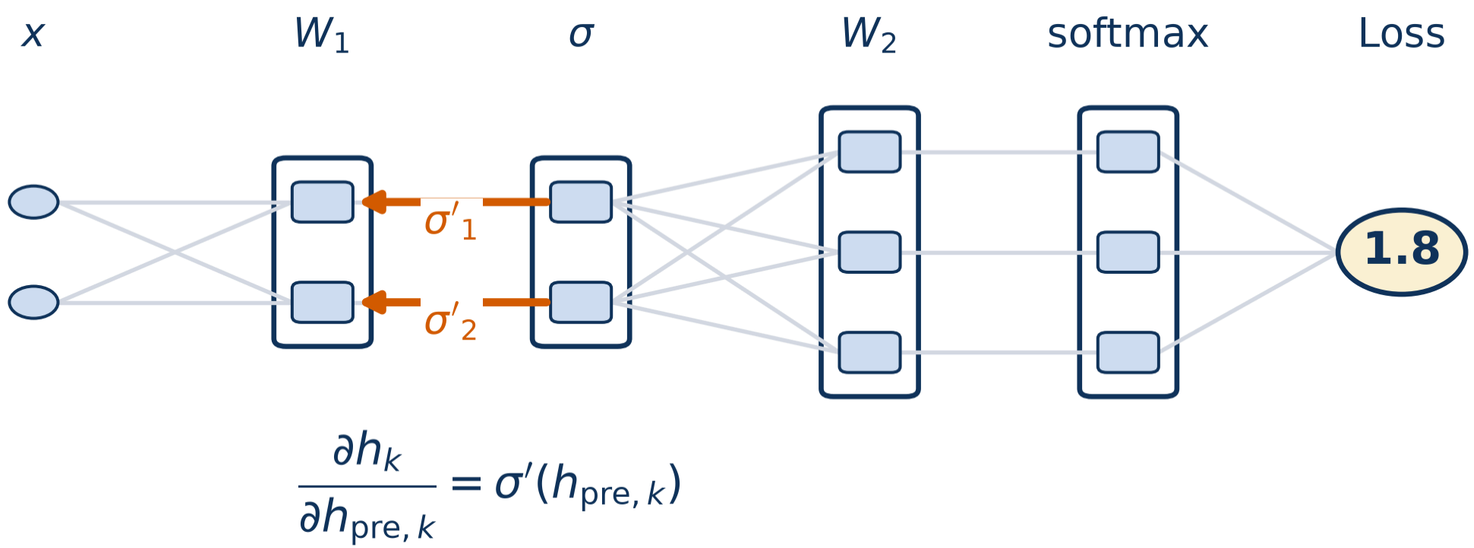
Passing the error to h_1: how should it change?



$$\frac{\partial L}{\partial h_1} = \sum_j (\hat{y}_j - y_j)\, w_{2,j1}$$

- Same wires as the last slide, but now asking how h_1 should change rather than the weights. Here $w_{2,j1}$ is column 1 of W_2: the weights leaving h_1
- h_1 feeds every output neuron, so it collects blame from all of them, each weighted by $w_{2,j1}$; in matrix form that sum is $(W_2)^\top(\hat{y} - y)$

Through the activation



- Each hidden value's error is scaled by its own neuron's activation slope, independently of the others
- Written for the whole layer: $\frac{\partial L}{\partial h_{\text{pre}}} = \frac{\partial L}{\partial h} \odot \sigma'(h_{\text{pre}})$ ($\odot$: elementwise)

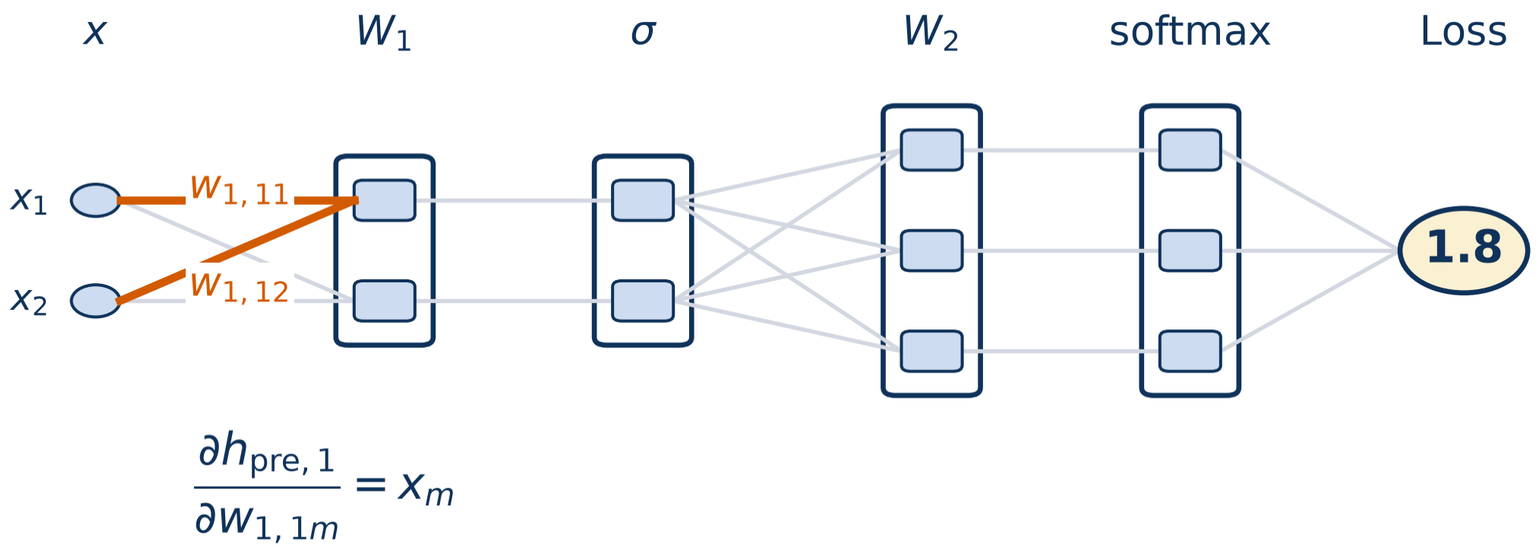
ReLU’s derivative

- For ReLU, the slope is especially simple:

$$\sigma'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

> **Important**
>
> If a neuron’s pre-activation was ≤ 0, ReLU zeroed it going forward — and now its slope is 0 too, so **no gradient passes through it at all**, no matter how large the incoming error was

The hidden layer's weights



- Same move as the output layer: the weight $w_{1,1m}$ (orange) multiplies the input x_m that feeds it, so $\dfrac{\partial h_{\text{pre},1}}{\partial w_{1,1m}} = x_m$
- Chain it on: $\dfrac{\partial L}{\partial w_{1,1m}} = \dfrac{\partial L}{\partial h_{\text{pre},1}} \cdot x_m$

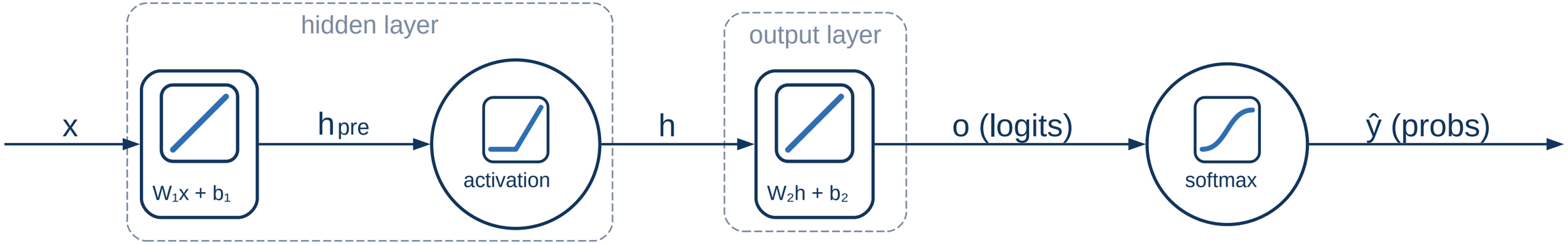
The same rule, in matrix form

quantity	matrix form
$\dfrac{\partial L}{\partial o}$	$\hat{y} - y$
$\nabla W_2,\ \nabla b_2$	$(\hat{y} - y)\, h^\top,\ \ \hat{y} - y$
$\dfrac{\partial L}{\partial h}$	$(W_2)^\top (\hat{y} - y)$
$\dfrac{\partial L}{\partial h_{\text{pre}}}$	$\dfrac{\partial L}{\partial h} \odot \sigma'(h_{\text{pre}})$
$\nabla W_1,\ \nabla b_1$	$\dfrac{\partial L}{\partial h_{\text{pre}}}\, x^\top,\ \ \dfrac{\partial L}{\partial h_{\text{pre}}}$

Every line is “the error flowing into a layer, times what fed into it” — the same local rule, once per layer, moving backward

Worked example: the backward pass

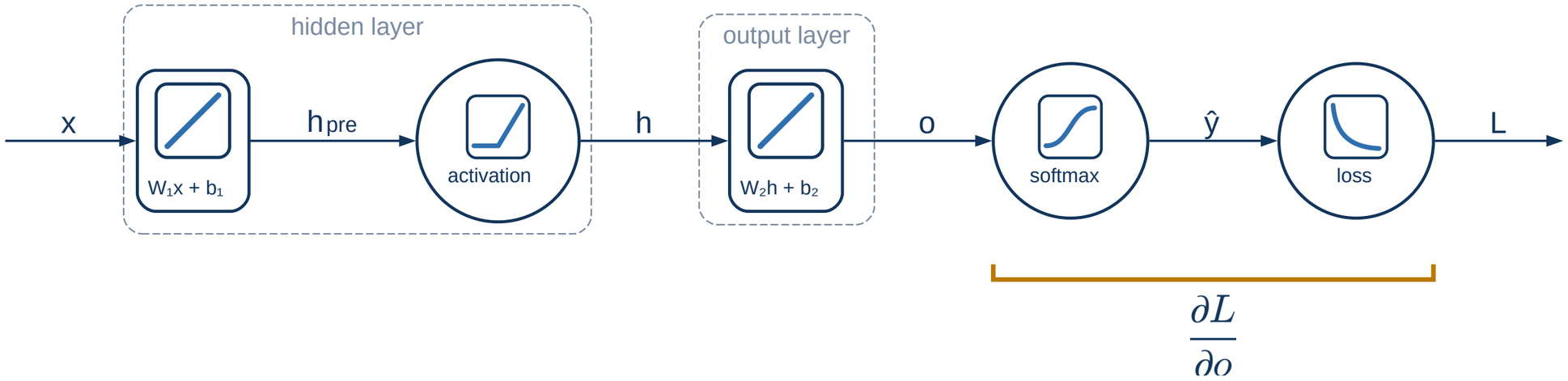
Setup, recalled from last lecture



x	h_{pre}	h	o	$\hat{y}$
$[1.2,\ 0.2]$	$[1.4,\ -0.8]$	$[1.4,\ 0]$	$[0.7,\ -0.42,\ 0.28]$	$[0.50,\ 0.16,\ 0.33]$

True class: banana, so $y = [0,\ 1,\ 0]$ and $L \approx 1.8048$

Step 1: start at the loss



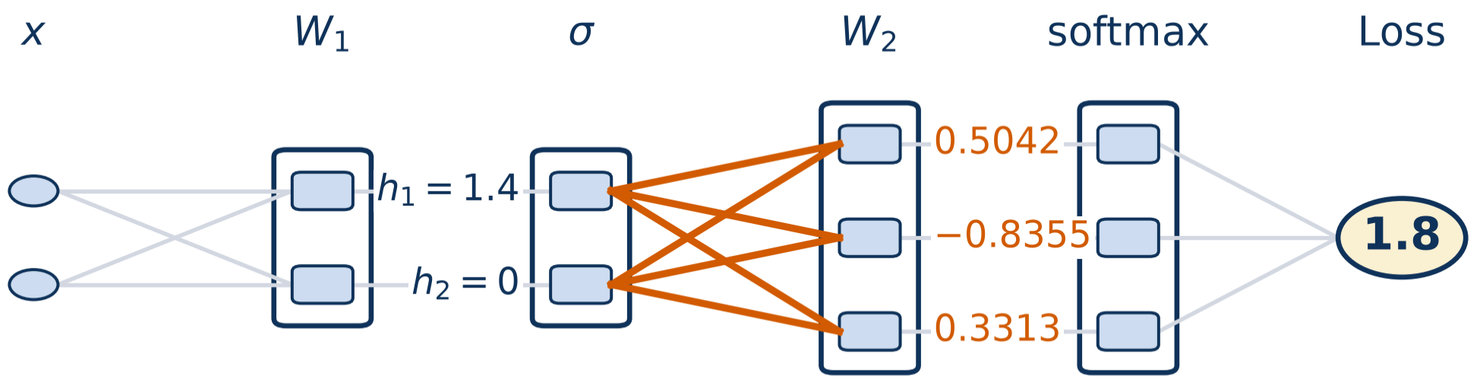
$$\frac{\partial L}{\partial o} = \hat{y} - y$$

$$= [0.5042,\ 0.1645,\ 0.3313] - [0,\ 1,\ 0]$$

$$= [0.5042,\ -0.8355,\ 0.3313]$$

- We subtract the whole vector y: only banana's entry loses a 1 (the others subtract 0). Negative means push that logit up, positive means push it down

Step 2: how should the output weights change?

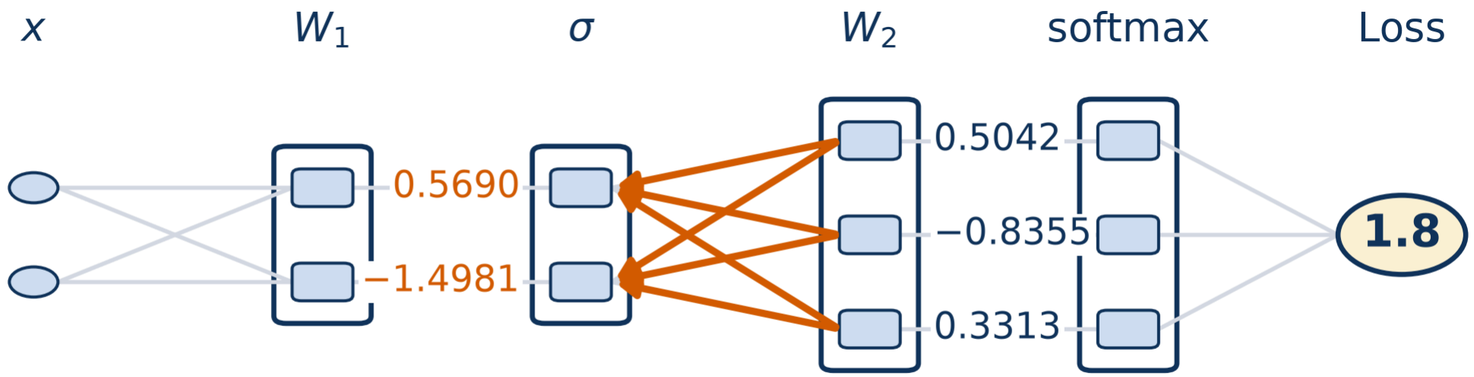


left: the value fed in (h) right: the error at each output ($\hat{y} - y$)

$$\nabla W_2 = \begin{bmatrix} 0.5042 \\ -0.8355 \\ 0.3313 \end{bmatrix} \begin{bmatrix} 1.4 & 0 \end{bmatrix} = \begin{bmatrix} 0.7059 & 0 \\ -1.1697 & 0 \\ 0.4638 & 0 \end{bmatrix}$$

- Each weight's gradient is the error at its output neuron times the value that fed it. Column 2 is 0 because $h_2 = 0$; ∇b_2 is just the error vector

Step 3: passing the error back to h



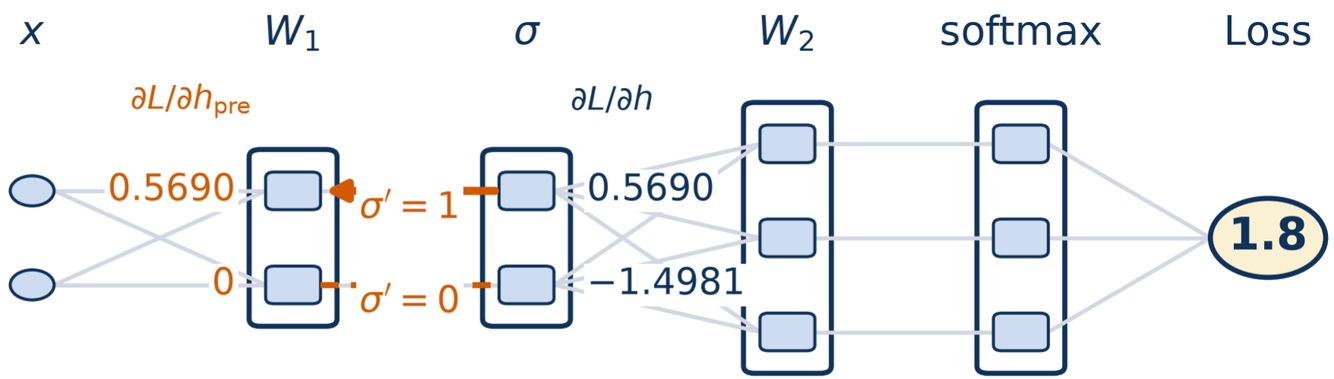
right: the error at each output left: the error arriving at each h_k

$$\frac{\partial L}{\partial h_1} = (0.5)(0.5042) + (-0.3)(-0.8355) + (0.2)(0.3313) = 0.5690$$

$$\frac{\partial L}{\partial h_2} = (1)(0.5042) + (2)(-0.8355) + (-1)(0.3313) = -1.4981$$

- Each output neuron sends its error back along its weights, and h_k adds up what arrives

Step 4: back through ReLU



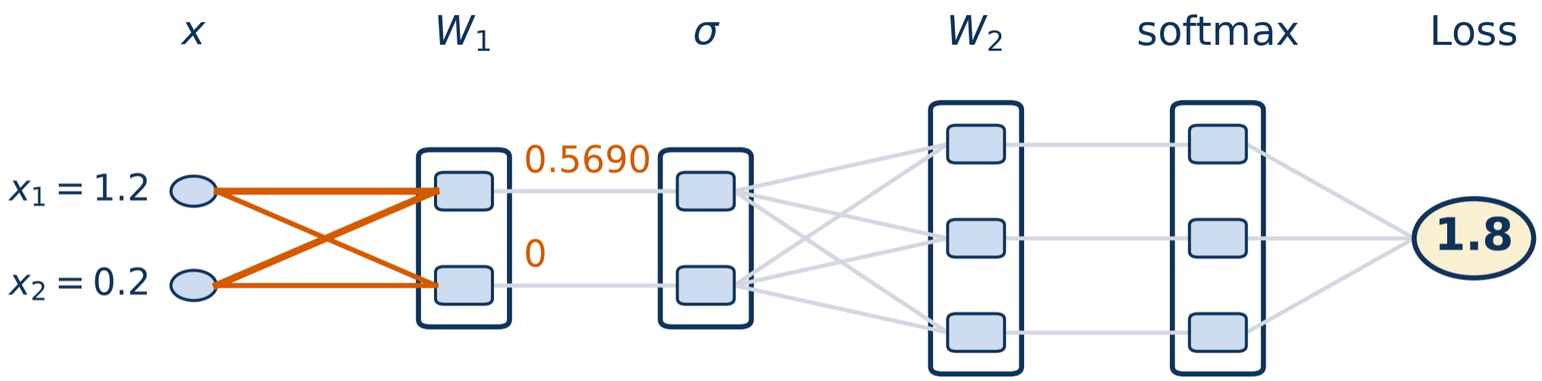
on the wires: ReLU's slope σ', which multiplies each error; a slope of 0 blocks it

$$\frac{\partial L}{\partial h_{\text{pre},1}} = 0.5690 \times \sigma'(1.4) = 0.5690 \times 1 = 0.5690$$

$$\frac{\partial L}{\partial h_{\text{pre},2}} = -1.4981 \times \sigma'(-0.8) = -1.4981 \times 0 = 0$$

- ReLU’s slope is 1 for positive inputs and 0 for negative, so h_2’s error is discarded

Step 5: how should the hidden weights change?



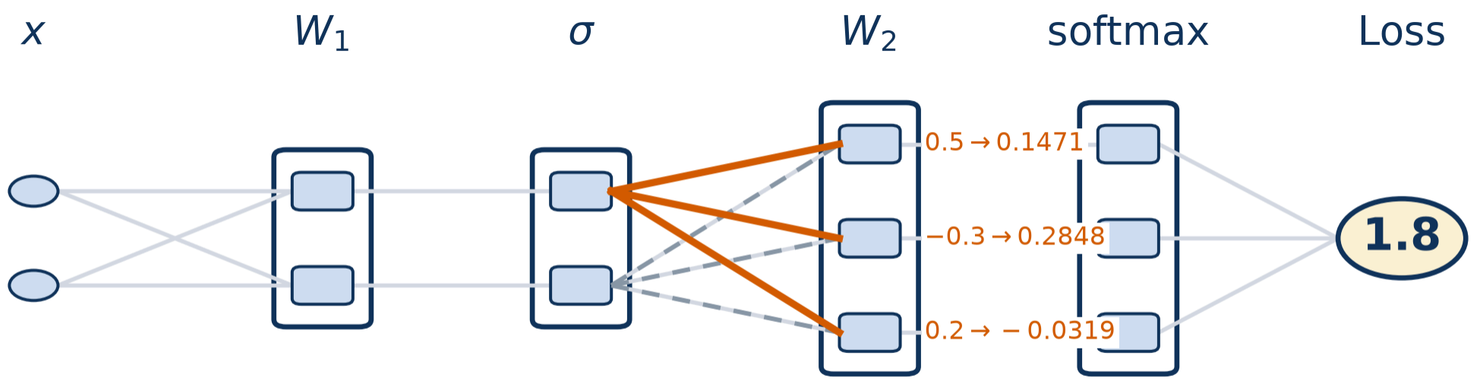
right of W_1: the error at each hidden neuron left: the input x

$$\nabla W_1 = \begin{bmatrix} 0.5690 \\ 0 \end{bmatrix} \begin{bmatrix} 1.2 & 0.2 \end{bmatrix} = \begin{bmatrix} 0.6828 & 0.1138 \\ 0 & 0 \end{bmatrix}$$

- Same move as Step 2: error at each hidden neuron times the input that fed it. Row 2 is 0 because its error was discarded at ReLU; $\nabla b_1 = [0.5690,\ 0]$

Updating the parameters

Updating the output layer



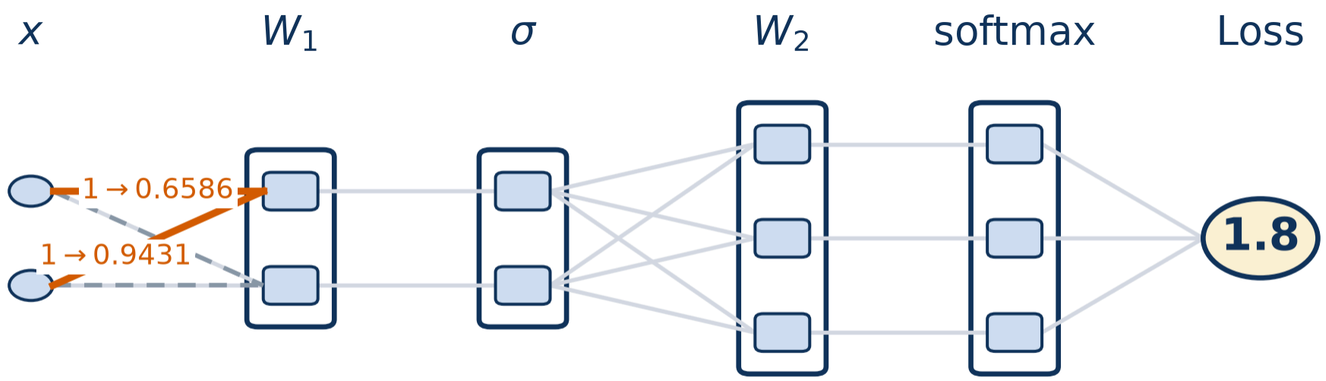
orange: the weights from h_1 (old → new) gray: unchanged

$$W_2 - 0.5\,\nabla W_2 = \begin{bmatrix} 0.5 & 1 \\ -0.3 & 2 \\ 0.2 & -1 \end{bmatrix} - 0.5 \begin{bmatrix} 0.7059 & 0 \\ -1.1697 & 0 \\ 0.4638 & 0 \end{bmatrix} = \begin{bmatrix} 0.1471 & 1 \\ 0.2848 & 2 \\ -0.0319 & -1 \end{bmatrix}$$

$$b_2 - 0.5\,\nabla b_2 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - 0.5 \begin{bmatrix} 0.5042 \\ -0.8355 \\ 0.3313 \end{bmatrix} = \begin{bmatrix} -0.2521 \\ 0.4177 \\ -0.1656 \end{bmatrix}$$

- Column 1 of W_2 changes; column 2 doesn't, because $h_2 = 0$ gave it a gradient of 0

Updating the hidden layer



orange: the first neuron's weights (old → new) gray: unchanged

$$W_1 - 0.5\,\nabla W_1 = \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix} - 0.5 \begin{bmatrix} 0.6828 & 0.1138 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0.6586 & 0.9431 \\ -1 & 2 \end{bmatrix}$$

$$b_1 - 0.5\,\nabla b_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.5 \begin{bmatrix} 0.5690 \\ 0 \end{bmatrix} = \begin{bmatrix} -0.2845 \\ 0 \end{bmatrix}$$

- Row 1 of W_1 changes; row 2 doesn't, because its error was blocked at ReLU

Did the update help?

Run the same forward pass again, with the updated parameters:

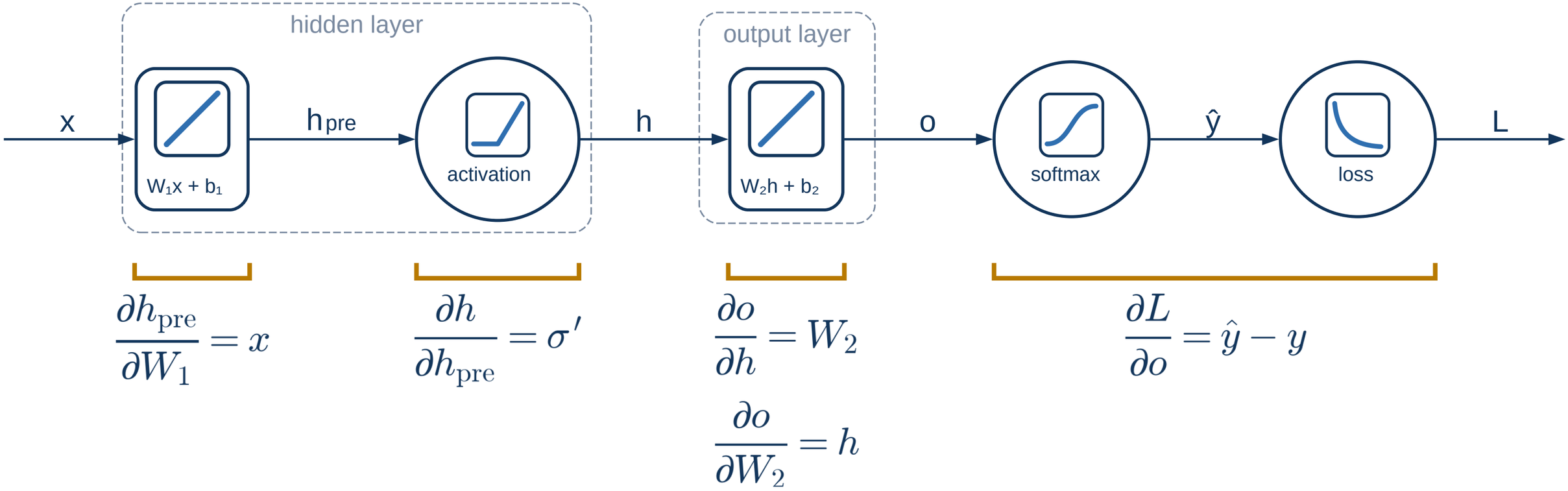
$$o_{\text{new}} = [-0.15,\ 0.6155,\ -0.1878], \qquad \hat{y}_{\text{new}} = [0.2431,\ 0.5228,\ 0.2341]$$

$$L_{\text{new}} = -\log(0.5228) \approx 0.6486$$

- Loss went from **1.8048 → 0.6486** — a bigger drop than the single-layer classifier managed
- And banana now **wins**: 0.5228 is the highest probability, up from third place. One step, two layers, already flipped the decision

Summary

Backpropagation, in one picture



- Multiply the local derivatives along the path from the loss back to each weight — the chain rule, one stretch at a time

Study guide: backpropagation

Needs	the values from the forward pass: $h_{\text{pre}}, h, o, \hat{y}$
Computes	the gradient of the loss for every weight and bias
How	multiply local derivatives from the loss backward, one stretch at a time
Then	update every parameter: $\theta \leftarrow \theta - \eta \nabla\theta$
Works for	any number of layers: one more stretch per layer
Watch out	a ReLU slope of 0 blocks the error; very long chains of products can shrink it

Tip

In lab: walk this exact example's forward and backward pass **by hand**, including the blocked error